The Norns III: Seeding Simulated-Annealing Placement with Provenance and Recovered Clusters How hot can the anneal start before it destroys a good initial placement?

Peter Jamieson
Electrical and Computer Engineering, Miami University
Oxford, OH, USA
jamiespa@miamioh.edu

Abstract—Simulated annealing is the workhorse of FPGA placement, and it starts from an initial placement that the anneal then destroys and rebuilds. What if the initial placement carried real information – the HDL hierarchy a provenance-tracked front end kept, or the clusters a graph analysis recovered from the flat netlist? This paper asks whether such seeds improve placement, and, because the anneal's early phase can erase any seed, at what initial temperature the seed's advantage dies. The seeds come from Odin III's provenance hierarchy and from Loki III's L2 clusterings (Louvain, spectral, affinity propagation, k-means); the die-level arrangement of clusters comes from a cut-weighted quotient-graph layout; and the seeds enter VPR through its initial-placement and floorplanning-constraint interfaces, with VPR's stock checkpointing flag (on by default, following RLPlace) enabled throughout and an off sweep as the control arm. We report seed retention at scale 1 and at twice each design's crossover, quality of results against an unseeded baseline, and the largest tested temperature scale at which each seed still helps – the recipe a clustering-based initial placement should be used with. A second round adds bitwise seeds (Odin III word membership laid out as bit-sliced runs), a random start and a coarse-random arm, and two structure analyses that need no placement run; each hypothesis is reported with its deciding number, and an appendix keeps the research log.

Index Terms—FPGA placement, simulated annealing, provenance, clustering, VTR

I. Introduction

VPR's simulated-annealing placer starts from an initial placement and anneals it: the schedule begins hot enough to move anything, and the early phase rearranges the circuit wholesale. The question this paper asks is whether the initial placement can be made to mean something, and how much of that meaning survives the anneal. Gharibian et al. showed that system-level structure recovered from a flat netlist correlates with placement locality [1]–[3]. Odin III keeps the provenance link that makes recovery unnecessary for the hierarchy [1], and Loki III recovers clusters from the flat netlist without a language model. RLPlace [4] showed that the anneal itself can be improved – directed moves chosen by a reinforcement-learning agent and checkpointing that prevents oscillation – starting from VTR's analytic placement. This paper connects those lines: the seed carries the structure, the anneal is measured for how much of it the seed can keep, and the largest tested temperature scale at which the advantage survives is reported as data, not tuned away.

The contributions are: (1) a seeding methodology – provenance hierarchy and four recovered clusterings, arranged on the die by a quotient-graph layout, entering VPR through its initial-placement and constraint interfaces, with VPR's stock checkpointing enabled and its off sweep as the control arm; (2) the crossover sweep, which calibrates the highest useful initial temperature per design against the best placement any seed achieves; (3) the retention and quality numbers per seed, with the full-seed versus inter-cluster-only split, which measures sensitivity to the intra-cluster order; (4) a second round that adds bitwise (word and bit-slice) seeds, a random-start and a coarse-random arm that separate the clustering from the coarse layout, and structure analyses of when seeding helps (Sections III-G and IV-A), with the research log in the appendix.

II. Background

A. Simulated annealing placement and RLPlace

VPR places by simulated annealing from an analytic initial placement [5], [6]. The anneal's moves are mostly random; its early phase can undo any structure the start carried. RLPlace improves the anneal rather than the start: a K-armed bandit selects among directed move types (simple moves and swaps, bounded centroid, median, critical-path moves), and checkpointing periodically saves the best placement so oscillation cannot throw it away [4]. Our experiment keeps the anneal as-is (VPR already ships RLPlace-style checkpointing, on by default) and changes the start instead; the checkpointing prediction – that save-best/restore extends the useful temperature range – is one of the hypotheses we test.

B. The seeds

The provenance hierarchy (Urd) is the module structure Odin III records on every netlist object; it needs no recovery. Urd has a single cluster only on sha and stereovision3, two on diffeq1 and diffeq2 (their one module plus a flat group), and one per populated module plus a flat group (3 to 23) on the other designs, so its signal is degenerate on the two one-cluster designs. The recovered clusters (Verdandi) are Loki III L2 partitions: Louvain, spectral, affinity propagation and k-means at several k, scored in the Loki III paper [7] against the known hierarchy. Agent default: Norns builds the Loki L0 graph.json from the same Odin III BLIF, because Loki's key.json uses Odin API names that do not join to VPR atom names; the clustering code is Loki's. The two seed families differ exactly in the question's terms: provenance is the structure the designer wrote, clusters are the structure the flat netlist still shows.

III. Methodology

A. Seed construction

Each seed is a partition of the placed blocks. Its die-level arrangement comes from the quotient graph: contract each cluster to a node, weight edges by inter-cluster net cuts, lay the quotient out by a coarse force-directed solve, and scale it to the die. Inside a cluster, blocks take the seed's internal order, or a shuffle in the inter-cluster-only arm; the two arms differ only in that intra-cluster order, not in the coarse cluster layout.

B. Seed construction in detail

One die, one netlist. Every seed targets the same die: VPR's k6_frac_N10_frac_chain_mem32K_40nm architecture with six-input LUTs (K = 6). The netlist is the VTR-dialect BLIF that Odin III's front end writes for each design: read_verilog –nolatches –vtr-compat, proc, fsm, opt, flatten, opt -full, wreduce, hardmap, lower, abc –lut 6, stats, and write_blif –vtr –lut-k 6 –name-style provenance. Each configuration is one deterministic VPR run (–seed 1). The unseeded baseline run is the comparison point and also supplies the seed builders: its analytic initial placement (initial.place) gives every soft seed legal slots and its fine order, and its final.flat_place is the reference the retention metric uses.

Coarse layout. Every soft seed shares one die-level arrangement, the quotient-graph layout. Each cluster is contracted to one node and each pair of clusters is weighted by the number of nets cut between them; the weighted quotient graph is laid out by a force-directed solve (networkx.spring_layout, fixed seed), and the embedding is scaled onto the die's logic-slot bounding box. Each cluster then takes the free CLB slots nearest its laid-out point, nearest first. Clusters that exchange many nets therefore start close together, and the die fills in cluster-sized groups instead of a scatter.

Fine order. The soft arm has two variants that differ only inside a cluster. Full keeps the baseline analytic order: a cluster's blocks keep their relative analytic positions when they are assigned the cluster's slots. Inter shuffles the intra-cluster order under a fixed shuffle seed, so only the inter-cluster structure survives; because the coarse cluster layout is identical in the two variants, the pair isolates the fine order. The hard arm has no fine-order choice: it hands VPR one PartitionRegion per cluster, sized from the full seed's slots and expanded until it holds the cluster's logic (and nudged to cover a memory or multiplier site the cluster needs), and VPR's placement orders the blocks inside each region.

Placement macros. Every seed honours the die's placement macros: I/O pads never move (their I/O sites are fixed by the architecture), mult_36 macros stay anchored on their tile footprint, and an adder carry chain moves as one vertical run in a single column, head at the top (VPR places a chain as a direct vertical macro). Chains are assigned slots before the nearest-first fill, so a chain is never split.

The seed families. The Urd seed is the provenance hierarchy made explicit: Odin III writes each block's HDL module into its netlist name, the seed builder reads that module from the BLIF, and each populated module becomes one cluster; blocks that carry no module name join one flat cluster. Urd is one cluster on sha and stereovision3, two on diffeq1 and diffeq2 (one module plus the flat group), and one per populated module plus the flat group on the other designs. The rationale is the designer's locality: modules are the units the designer grouped, and logic inside a module was written to communicate (shared words, state, control), so a module cluster concentrates the connectivity the placer is paid to shorten. That is the hypothesis the Urd arm tests.

The Verdandi seeds are Loki III's L2 clusterings of the flat netlist: Louvain, spectral, affinity propagation, and k-means at k, $2k$ and $4k$, with k the design's module count (two where the design has one module). Norns builds the L0 graph.json the clusterings read from the same Odin III BLIF, because Loki's key.json uses Odin API names that do not join to VPR atom names; the clustering code is Loki's. A recovered cluster approximates the provenance grouping from connectivity alone, so comparing Urd with Verdandi isolates what the HDL structure adds over the graph.

Why a seed should beat a random start. A random placement scatters communicating logic across the die; the quotient layout puts each cluster's logic together and puts clusters that exchange nets next to each other, so the anneal starts from the structure it is meant to find and refines from there. The experiment tests that mechanism rather than assuming it, and it compares against VPR's analytic start, which already respects connectivity, not against a random placement. Two measurements report what the seed did: retention, the fraction of a seed's CLBs found inside its seed cluster's region at exit, has medians 0.68 (full) and 0.70 (inter) at VPR's own start temperature

and 0.52–0.74 per family at twice the crossover; and the crossover sweep finds the largest tested temperature scale at which the seed still wins, the point where its contribution stops surviving the anneal.

What the HDL constructs say about a good start. HDL modules, buses, datapath words and finite-state machines group logic that communicates: a bus is one word fanned out to every consumer, a datapath chain is a dataflow line, and an FSM's next-state logic and registers exchange signals every cycle. Provenance preserves that grouping in the netlist, so a seed built from it starts the placer with the designer's locality already on the die; L2 clusters try to rediscover it from connectivity alone, and this experiment measures how well that substitute places, not how closely the clusters match the modules. That is why Urd and Verdandi are the arms, and why the paper asks whether recovered structure is interchangeable with provenance.

C. Injection

Seeds enter VPR two ways, both already in the pinned build (v8.0.0+18416): soft, as an initial placement file (–read_initial_place_file) that the anneal is free to move away from; and hard, as floorplanning constraints (–read_vpr_constraints) with one PartitionRegion per cluster, which the anneal cannot violate at any temperature. The hard arm constrains the regions only, so the anneal chooses the fine placement inside each one (one hard variant, no full/inter split). –write_initial_place_file dumps VPR's own analytic start for the baseline study. VPR's placement macros are respected: I/O pads and the multiplier tiles stay anchored, and carry chains move as one vertical run. The seeds are built from an Odin III provenance BLIF and Loki III's clusterings of the same netlist, arranged by a cut-weighted quotient-graph layout with networkx (spring layout, fixed seed). Building the seed files also needs VPR's unseeded baseline placement for legal slots, so each design's baseline run precedes its seeds.

D. The crossover sweep

The calibration loop follows the data, in order. First, run every seed and keep the best final placement P^* (and which seed made it). Second, cold-start a fresh anneal from P^* and sweep the initial temperature upward; the crossover is the largest tested scale at which the cold start still routes shorter than the unseeded run at the same scale, a lower bound on the true fall point when the sweep stops at its cap. The sweep is run once per design, so S is design-level and identical across that design's seeds. The temperature is reported as a multiple S of VPR's automatic start-temperature estimate. Third, run every seed at twice the design's crossover against an unseeded run at the same temperature: whether the seed still wins there tests how much of its advantage survives a hot restart. The checkpointing arm repeats the sweeps with VPR's checkpointing disabled; per RLPlace, the crossover should move hotter when it is on.

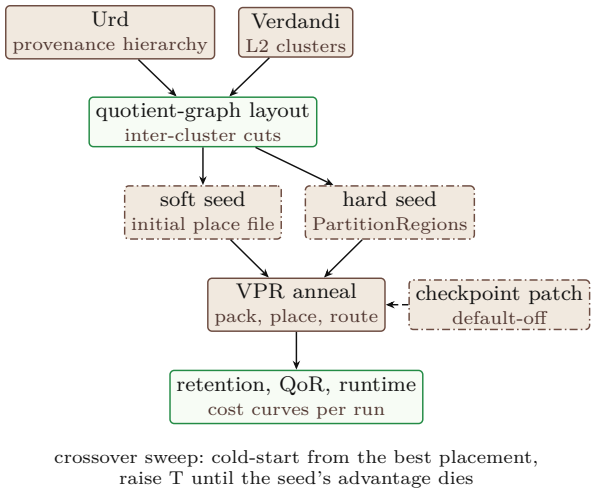


Fig. 1. The Norns III experiment. Seeds from provenance and recovered clusters, arranged by a quotient-graph layout, enter VPR softly (initial placement) or hard (constraints). The crossover sweep cold-starts from the best placement and raises the initial temperature until the seed's advantage dies or the sweep reaches its cap; the checkpointing arm (VPR's stock flag, on vs off) tests whether save-best/restore extends the range.

E. Metrics and data

Per run: seed retention (fraction of cluster CLBs still inside their seed cluster's region at exit; mean displacement from the seed coordinate), routed wirelength, critical-path delay, minimum channel width, placement runtime, and the per-temperature cost curve. Designs are the 15 VTR benchmarks that fit the development machine, smallest first, one run at a time under the heavy lock. Every number in this paper traces to a kept record; large data keeps its scripts.

F. Benchmarks

Table I reports each design's construction and its post-technology-mapping size. The module count is the front end's, while the CLB count and the grid are VPR's own numbers for the packed netlist, so the table describes the designs as the experiment actually places them.

The 15 designs span two orders of magnitude after technology mapping: from 13 CLBs (ch_intrinsics) to 1956 (stereovision2), 4843 CLBs in total. The mean is 322.9 CLBs and the median 125; the mean sits well above the median because three stereovision designs carry most of the logic. VPR auto-sizes a square grid to the packed netlist, with a mean side of 27.2 tiles (median 20) and a mean of 1076.4 grid tiles (median 400). Grid size follows the largest resource demand, not the CLB count: I/O-heavy and memory-heavy designs such as mkDelayWorker32B get a large die for few CLBs.

TABLE I
Benchmark construction and post-technology-mapping size. All fifteen designs are from the VTR 8.0 benchmark set (vtr_flow/benchmarks/verilog). Modules is the front end's module count, CLBs the clb blocks VPR packs, Memory and Mult the memory and mult_36 blocks, and Grid the auto-sized device grid with its tile count in parentheses. Sizes are read from each design's baseline run log and kept in data/benchmark-sizes.json.

Design	Suite	Modules	CLBs	Memory	Mult	Grid (tiles)
ch_intrinsics	VTR 8.0	2	13	1	0	10×10 (100)
stereovision3	VTR 8.0	1	15	0	0	7×7 (49)
diffeq2	VTR 8.0	1	22	0	5	16×16 (256)
diffeq1	VTR 8.0	1	37	0	5	16×16 (256)
mkPktMerge	VTR 8.0	7	21	15	0	28×28 (784)
raygentop	VTR 8.0	15	143	1	6	19×19 (361)
sha	VTR 8.0	1	142	0	0	17×17 (289)
boundtop	VTR 8.0	13	54	1	0	15×15 (225)
mkSMAdapter4B	VTR 8.0	8	125	5	0	20×20 (400)
or1200	VTR 8.0	16	252	2	1	27×27 (729)
stereovision0	VTR 8.0	25	707	0	0	33×33 (1089)
stereovision1	VTR 8.0	16	788	0	43	40×40 (1600)
mkDelayWorker32B	VTR 8.0	16	74	43	0	50×50 (2500)
stereovision2	VTR 8.0	24	1956	0	192	82×82 (6724)
blob_merge	VTR 8.0	2	494	0	0	28×28 (784)

G. Round two: bitwise seeds and structure analyses

Round one seeded VPR with module-level clusters. Datapath logic is bit-level: a 32-bit register or adder is a regular stack of identical bit cells, and the carry chain already shows that VPR places such stacks as ordered vertical runs. Round two therefore adds a bit-level seeding aspect that keeps the module hierarchy at the coarse level (Section A records where the idea came from).

a) Word membership from Odin III data, not names.: The obvious source, the provenance BLIF's block name module/$nN@file:line, is too coarse: the line is the HDL statement, so one always block is one group (the largest (module, file:line) group in any design holds up to 1739 blocks, and sha's holds 1739 of its 2506 blocks; 59 of the 161 statement lines that hold word blocks hold more than one word, up to 51). The finer records exist in the Odin III session that writes the BLIF, so the seed builder reruns the identical script, checks that the BLIF it writes equals the kept one byte for byte (all 9 designs match), and reads the records Loki III's labeller reads: the HDL name bus[i] of a cell's output net and the source spans of each cell, matched to the word-level cells that exist before technology mapping. A final cell joins its BLIF block through the net id the BLIF name carries ($n<id>), so the join is by identifier, not by string (241 of 11198 cells, 2.2%, found no BLIF block and are left unlabelled; the writer legalises dotted names, handled explicitly). A block's word is the first rule that applies: (bus) an output net named bus[i] puts it in word bus at bit i, which holds for every flip-flop that keeps a register name; (cell) exactly one word-level cell's spans lie inside the block's spans, which is true for adders, multipliers, memories and single-statement LUTs; otherwise (none) the block has no word. Bit order inside an adder word is the carry-chain position. ABC's cone provenance merges the spans of a LUT that mixes words, and its edge budget makes some runs share one record, so 21% of the LUTs get a word against 100% of the flip-flops and 99% of the hard cells. Over all designs, 58.7% of the blocks are in a word (per-design median 63.1%); 444 words in all.

b) Seed families.: A CLB joins a word cluster when at least one third of its atoms carry a word and that word is the plurality; otherwise it keeps its Urd module cluster. Four families follow, each entering VPR as a soft initial placement at VPR's own start temperature (scale 1). words: the word CLB groups are the clusters, arranged by the round-one quotient-graph layout. slices: the same clusters, but each word is laid out as a vertical run in bit order (bit 0 on top, the carry-chain direction), runs of equal length aligned to the same rows, so that bit i of connected words shares a row; true carry chains are untouched. hier: Urd module clusters at the coarse level and, inside each module, the words as bit-sliced runs. words_partial: only the word CLBs are seeded, every other CLB keeps VPR's analytic position (and every carry chain with an unseeded CLB stays whole). The words family also has the inter-cluster-only variant. Pads and multipliers stay anchored as in round one.

c) Random and coarse-random arms (RQ-N4).: random is a random legal start: every logic CLB takes a random slot and carry chains a random window, fed as a soft seed at scale 1. The coarse-random arm keeps the clusters and their within-cluster order but permutes the cluster positions on the die (urd_shuf for Urd, and the *_shuf twin of the best new family).

d) Structure analyses (no VPR runs).: RQ-N1 measures, on each design's unseeded baseline, how module- and word-coherent three placements are: VPR's analytic start, its final placement, and a random null (logic-CLB coordinates permuted, 20 samples). Coherence is the NMI between a $g \times g$ grid partition ($g = \lceil\sqrt{\#\text{labels}}\rceil$) and the labels, and the compactness (label CLBs per grid tile of the

TABLE II
Geomean quality ratios vs the unseeded baseline, per seed family, for the soft initial-placement arm; the inter-cluster-only row shuffles the order inside each cluster and pools all seven seed families, and the k-means row pools k, $2k$ and $4k$. Below 1.0 is better. The hard-constraint arm is discussed in the text.

Seed	Wirelength	Delay	Chan. width	Runtime
Urd (hierarchy)	1.029	1.004	0.958	1.068
Louvain	1.008	0.982	0.980	1.091
Spectral	1.006	0.993	0.968	1.098
Affinity propagation	1.002	1.003	0.982	1.086
k-means (k, $2k$, $4k$)	1.006	0.985	0.975	1.088
Inter-cluster only	1.015	0.981	0.987	1.102

TABLE III
The crossover sweep, per seed family: the largest tested scale S of VPR's automatic start-temperature estimate at which the cold start from P^* (the best placement over all seeds) still routes shorter than an unseeded run at the same S, and the seed's median retention at the hot step ($2S$). S is design-level (one sweep per design), so it is identical across families; where the sweep stopped at its cap, S is a lower bound.

Seed	Crossover S	Retention
Urd (hierarchy)	16×	0.744
Louvain	16×	0.520
Spectral	16×	0.635
Affinity propagation	16×	0.564
k-means (k, $2k$, $4k$)	16×	0.667

label's bounding box, CLB-weighted). RQ-N2 correlates the seed gain with three structure measures of the netlist graph: Louvain modularity Q; the z-score of Q against a degree-preserving null (double edge swaps, 20 samples); and the inverse Rent exponent $-p$, with p fitted from the terminals of BFS balls of geometric sizes (a lower exponent is a more local design). Spearman's ρ and p-value are reported with $n = 9$, which is small.

e) Hypotheses and decision rules.: Paired comparisons are per design (variant over reference), reported as a geomean ratio, the number of designs where the variant is better and a two-sided exact sign test. H7a: words beats Urd in soft wirelength (supported if the geomean is below 1 and $p < 0.05$, not supported if the geomean is at least 1, else inconclusive). H7b: slices beats words on wirelength and channel width (supported if both geomeans are below 1 and the wirelength sign test has $p < 0.05$). H7c: hier has the lowest geomean wirelength of all eleven soft families. H7d: words_partial is at least as good as the full word seed (supported if the geomean ratio is at most 1, inconclusive up to 1.01). H1: the baseline's final placement is module- and word-coherent (supported if, for both labellings, at least two thirds of the designs lie more than two null standard deviations above the null in both NMI and compactness). H2: structure predicts gain (supported if at least two of the three measures correlate positively with the best round-two gain at $p < 0.05$; not supported if no correlation reaches $p < 0.05$). H4: random start > permuted clusters > quotient layout ≥ analytic start in wirelength ratio, for Urd and for the best new family.

IV. Results

The 15 completed designs give the ratios of Table II. Soft seeds stay within a few percent of the unseeded baseline in routed wirelength (1.009 for the full seed, 1.015 for the inter-cluster-only arm, both pooled over all seven seed families) while cutting the minimum channel width to 0.973 and costing 1.087 in placement runtime (both also pooled). The hard arm is worse on average (geomean 1.088 over the 104 runs that completed; 14 of those still beat the baseline, and stereovision3's hard geomean is 0.906), but not uniformly: one blob_merge k-means$_{2k}$ run is missing from the arm because VPR's initial placement could not fit 9 CLBs inside its regions (the constraints are too tight, not a packing failure). Fixed regions may block the anneal's fine moves across cluster borders – a hypothesis the aggregate numbers are consistent with. No single seed family dominates: Urd is the worst full seed in soft wirelength (1.029) and the best hard seed (1.062 against 1.075-1.120 for the recovered families). Retention at scale 1 is 0.68 of a seed's cluster CLBs (full) and 0.70 (inter).

TABLE IV
Round two: geomean ratios vs the unseeded baseline at scale 1 for the soft arms (Urd from round one for reference). The last column is the number of designs where the routed wirelength is below the baseline. WL is routed wirelength, Chan. the minimum channel width, Time the placement runtime, T_0 the anneal's start temperature that VPR derives from the starting placement. Below 1.0 is better (except T_0, a diagnostic).

Seed	WL	Delay	Chan.	Time	T_0	Better
Urd (round 1)	1.026	1.047	0.933	1.061	2.96	2/9
Words	1.013	0.997	0.945	1.257	5.83	3/9
Words, inter only	1.018	1.006	0.944	1.272	7.04	3/9
Slices	0.969	0.998	1.002	1.278	8.92	7/9
Hier	1.010	1.006	0.968	1.257	4.40	4/9
Words, partial	0.985	1.008	0.936	1.262	5.42	5/9
Random start	0.975	1.003	0.996	1.274	6.94	5/9
Urd, permuted	1.035	1.011	0.941	1.241	2.91	2/9

A. Round two results

RQ-N7, bitwise seeds (9 designs, one VPR seed each). H7a is inconclusive: words over Urd is 0.986 in wirelength, better on 5 of 9 designs (sign test $p = 1.000$). H7b is inconclusive: slices over words is 0.957 in wirelength (better on 7 of 9, $p = 0.180$) and 1.060 in channel width (better on 3 of 9). H7c is not supported: hier ranks 8 of 11 families at 1.010; the lowest is slices at 0.969. H7d is supported: words_partial over words is 0.973 (better on 6 of 9, $p = 0.508$). Ties (identical results) are left out of the sign tests: 0 for H7a, 0 for H7b and 0 for H7d. A family with nothing to rearrange reproduces its reference exactly (a design whose words are all single-CLB, or that has one module): slices equals words on 0 designs, hier equals Urd on 0 and words_partial equals words on 0.

TABLE V
RQ-N1: medians over designs of the grid-partition NMI and label compactness of the analytic start, VPR's final placement and a random null (20 permutations). Designs with a single module have no module label and are left out of that row.

Labels	n	NMI (start/final/null)	Comp. (start/final/null)
Module	13	0.25/0.23/0.10	0.28/0.26/0.10
Word	15	0.72/0.73/0.66	0.33/0.32/0.08

TABLE VI
RQ-N2: Spearman ρ (p) between per-design structure and seed gain (1 - wirelength ratio): Urd round one, best of seven round-one families, best new family. The inverse Rent exponent is $-p$.

Gain	Q	$z(Q)$	$-p_{\mathrm{Rent}}$
Urd, round one	-0.17 (0.533)	-0.45 (0.089)	0.12 (0.657)
Best round one	-0.09 (0.742)	-0.24 (0.390)	-0.16 (0.558)
Best new family	0.15 (0.700)	-0.37 (0.332)	0.43 (0.244)

The best new family by geomean wirelength is slices, so the coarse-random, hard and crossover arms below use it.

RQ-N4, clustering versus coarse layout. H4 is not supported. For Urd the wirelength ratios are random start 0.975, permuted cluster positions 1.035, quotient layout 1.026; for slices they are 0.975, n/a and 0.969. The analytic baseline is 1 by definition, so a random start above 1 means VPR's analytic start is better than a random one, and the gap between the permuted and the quotient arm is what the coarse layout is worth beyond the clustering itself.

RQ-N1, does VPR rediscover the hierarchy? H1 is inconclusive. Table V gives medians over designs: the final placement is above the random null in NMI on 8 of 13 designs for modules and 9 of 15 for words, and in compactness on 10 and 14 (more than two null standard deviations). Final and analytic placements are close, and the final NMI is at least the analytic one on 5 of 13 (modules) and 7 of 15 (words) designs. With many small labels the NMI of a random placement is itself high (word null 0.66), so compactness is the cleaner signal.

RQ-N2, does structure predict the gain? H2 is not supported: 0 of 9 correlations reach $p < 0.05$ (no multiple-comparison correction; $n = 15$ designs, so only a large ρ can be told from chance). Table VI lists Spearman's ρ (and p).

V. Discussion

Three observations. First, the seed families span a narrow band but are not interchangeable: provenance (Urd) is the worst full seed in the soft arm (1.029) and the best in the hard arm, so it is not consistently better or worse than recovered clusters. The full and inter-cluster-only arms differ by 0.6% in pooled wirelength (1.009 vs 1.015); round one has no arm that randomizes the coarse cluster layout; the round-two permuted-cluster and random-start arms (Section IV-A) measure it. Second, VPR's checkpointing does not separate the crossover in the measured range: the two sweeps find the same scale on 14 of the 15 designs where both found one, and the control extends it on stereovision0 (control 32 vs 16). RLPlace's prediction that checkpointing extends the useful range is therefore not observed on these designs; if it holds, its effect is smaller than the between-design variation. Third, the safe recipe is to cold-start from a cluster-derived placement at a temperature scale at or below the crossover, which is a multiple of VPR's own estimate; above it the seed's retention is still high per family (medians 0.52-0.74 at twice the crossover, individual runs 0.21-1.00) but it wins only 57 of 105 head-to-heads against an unseeded run at the same temperature – the coarse structure survives the anneal, the advantage does not.

Round two. The bitwise idea is tested in four forms, and the evidence is this: word clusters against module clusters are inconclusive (0.986); laying the words out bit-sliced against the quotient layout of the same clusters is inconclusive (0.957 in wirelength); the hierarchical combination is not supported as the best seed (rank 8 of 11); and seeding only the word cells is supported as good as seeding everything (0.973). The word membership available from Odin III covers the flip-flops, adders, multipliers and memories but only 21% of the LUTs, because ABC's cone provenance merges the spans of mixed logic, so the word seeds are datapath-register seeds more than whole-datapath seeds; a result against bitwise seeding here is a result about that coverage as much as about the idea.

One confound the scale-1 comparison cannot remove: VPR derives its start temperature from the starting placement, so a start that is worse than the analytic one is annealed hotter and, in these runs, longer. The geomean start temperature relative to the baseline is 6.944 for the random start, 2.963 for Urd and 8.917 for the best new family, and placement runtime moves with it (Table IV). The arms are therefore compared at VPR's own temperature, not at equal anneal effort, and the random start beating a connectivity-aware start would be a statement about the schedule as much as about the start; the effort curve (N3) is the experiment that separates them and stays open.

On the coarse-versus-fine question that round one left open, the arms give the answer for these designs: H4 is not supported, with Urd at random 0.975, permuted 1.035 and quotient 1.026. RQ-N1 explains part of the round-one parity: VPR's own final placement is already clearly more module- and word-compact than a random placement (H1 inconclusive), so a seed that says the same thing adds little. Whether the structure of the netlist predicts where a seed helps is H2, not supported. All of this is $n = 15$ designs, median 125 CLBs, one VPR seed per configuration; differences of a percent are within what a different VPR seed can move.

VI. Conclusion

Seeding VPR's anneal is worth it only as a calibrated cold start: at VPR's own start temperature the cluster layouts and the provenance hierarchy place at parity with the analytic start, but from a good final placement they keep winning when the anneal is restarted hotter, up to the largest tested scale that still wins – a lower bound on the fall point where the sweep hit its cap. The recipe is that crossover table, not a seed-specific tuning: start at or below it, and expect the seed to inform the coarse placement while the anneal supplies the fine one.

Round two adds bitwise seeds to the provenance hierarchy. At VPR's own start temperature the best of them, slices, is 0.969 of the baseline's wirelength, the Urd hierarchy is 1.026, and the question that made the idea worth testing, bit-level structure against module structure, comes out inconclusive (H7a) and not supported (H7c) on these designs.

Appendix

This log records the thought pattern of the study so a reader can follow how each result produced the next question. Entries are dated; numbers are those of the generated tables.

a) 2026-10-23, round one: the question and its hypotheses.: Does seeding VPR's anneal from netlist clusters or the provenance hierarchy improve placement, and how hot can the anneal start before it destroys the seed? Hypotheses: a seed helps at VPR's own start temperature; the helpful temperature range is bounded by a design-level crossover; the provenance hierarchy and recovered clusters differ; hard regions help or hurt.

b) 2026-10-24, what round one said.: Soft seeds sat at parity with the analytic baseline (wirelength 1.009 full, 1.015 inter-cluster only; channel width 0.973; runtime 1.087); hard regions were worse (1.088, 14 of 104 better); seed retention was 0.68/0.70 at scale 1 but a seeded run won only 57 of 105 head-to-heads at twice the crossover; Urd was the worst soft and the best hard seed.

c) 2026-10-24, what it suggested.: The baseline is VPR's connectivity-aware analytic start, so better than random was never tested; the anneal budget may erase what a seed adds (an effort curve is untested, N3); VPR's flat placement may already rediscover the structure the seed carries (N1); the designs are small, median 125 CLBs (size effect untested, N5); and whether the gain is the clustering or the coarse layout was not separable (N4).

d) 2026-10-24, Peter's bitwise idea.: "consider that bitwise is more relevant in the Norn path so we should consider that as an additional seeding aspect (not necessarily the entire HDL structure (though hierarchically yes)). Let's include this and run the next set of experiments noting this thought pattern in the appendix of each paper so we can see the progress ... write up into the papers findings." Reading: Norns is the datapath path, bits are the unit of structure there, so word and bit-slice structure is a seeding aspect of its own, with the module hierarchy kept at the coarse level.

TABLE VII
Research questions and hypotheses of the study with status and the deciding number.

RQ	Hypothesis	Round	Status	Deciding number
R1	soft seed stays at parity	1	supported	wirelength 1.009
R1	hard regions help	1	not supported	1.088 (14/104)
R1	seeds win at 2× crossover	1	not supported	57/105
N7	H7a words beat Urd	2	inconclusive	0.986 (5/9)
N7	H7b slices beat words	2	inconclusive	WL 0.957, ch. 1.060
N7	H7c hier is best	2	not supported	rank 8/11
N7	H7d partial $\geq$ full	2	supported	0.973
N4	H4 random $<$ permuted $<$ quotient	2	not supported	0.975/1.035/1.026
N1	H1 baseline already coherent	2	inconclusive	8/13 mod., 9/15 word
N2	H2 structure predicts gain	2	not supported	0/9 at $p < 0.05$

e) 2026-10-24, round two: questions and hypotheses.: N7 (bitwise seeds): H7a to H7d. N1: H1 (VPR rediscovers the hierarchy). N2: H2 (structure predicts the gain). N4: H4 (random start, permuted clusters, quotient layout, analytic start). The word membership had to come from data: the provenance BLIF line is a statement, so the membership is read from Odin III's bus names and source spans and joined by net id (Section III-G); only about 21% of the LUTs get a word.

f) 2026-10-24, round two: outcomes.: Inconclusive for H7a, inconclusive for H7b, not supported for H7c, supported for H7d, not supported for H4, inconclusive for H1 and not supported for H2; the numbers are in Table VII and Section IV-A.

g) Still open.: N3 (effort curve: how the result changes with the anneal budget, e.g. inner_num); N5 (size: designs of 1000 CLBs and up, where a seed has more to say); N6 (region looseness: constraint regions between the soft seed and the tight regions of round one); LUT coverage of the word membership (a front-end change, not a seeding change); more than one VPR seed per configuration.

References

[1] F. Gharibian, L. Shannon, and P. Jamieson, "Finding system-level information and analyzing its correlation to FPGA placement," in Proc. Int. Conf. Field Programmable Logic and Applications (FPL), 2010, pp. 544–549.

[2] F. Gharibian, L. Shannon, P. Jamieson, and K. Chung, "Analyzing system-level information's correlation to FPGA placement," ACM Trans. Reconfigurable Technology and Systems, vol. 6, no. 3, pp. 15:1–15:21, 2013.

[3] F. Gharibian, L. Shannon, and P. Jamieson, "Identifying and placing heterogeneously-sized cluster groupings based on fpga placement data," in Field Programmable Logic and Applications , 2014 24th International Conference on. IEEE, 2014, pp. 1–6.

[4] M. A. Elgammal, K. E. Murray, and V. Betz, "RLPlace: Using reinforcement learning and smart perturbations to optimize FPGA placement," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 41, no. 8, pp. 2532–2545, Aug. 2022.

[5] J. Luu, J. Goeders, M. Wainberg, A. Somerville, T. Yu, K. Nasartschuk, M. Nasr, S. Wang, T. Liu, N. Ahmed, K. B. Kent, J. Anderson, J. Rose, and V. Betz, "VTR 7.0: Next generation architecture and CAD system for FPGAs," ACM Trans. Reconfigurable Technology and Systems, vol. 7, no. 2, pp. 6:1–6:30, Jun. 2014.

[6] K. E. Murray, O. Petelin, S. Zhong et al., "VTR 8: High-performance CAD and customizable FPGA architecture modelling," ACM Trans. Reconfigurable Technology and Systems, vol. 13, no. 2, 2020, article 9.

[7] P. Jamieson, "Loki III: Reverse engineering structural netlists without a language model," 2026, manuscript.